

Real-Time Network Intrusion Detection System: Complete Project Roadmap

Phase 1: Project Planning & Requirements (2-3 weeks)

Week 1: Initial Planning

- Define project scope and objectives
- Identify key stakeholders and resources
- Conduct preliminary literature review on NIDS and ML applications
- Create project charter and timeline

Week 2: Requirements Gathering

- Define functional requirements for detection capabilities
- Establish performance metrics (detection rate, false positive rate, response time)
- Identify network integration requirements
- Determine data storage and processing needs
- Draft system architecture diagram

Week 3: Technical Specification

- Select technical stack and frameworks
- Define data collection strategy
- Outline ML model approach
- Create detailed system architecture
- Establish development environment

Phase 2: Data Collection & Environment Setup (3-4 weeks)

Week 4-5: Environment Setup

- Set up development environment with version control
- Configure network monitoring tools
- Set up data collection infrastructure
- Create containerized development environment
- Set up CI/CD pipeline for testing

Week 6-7: Data Collection & Processing

- Implement packet capture mechanisms
- Create feature extraction pipeline
- Establish data storage solution
- Develop data preprocessing modules

- Implement data augmentation techniques
- Create synthetic attack data as needed

Phase 3: Model Development (6-8 weeks)

Week 8-9: Baseline Modeling

- Develop initial anomaly detection models
- Implement feature selection algorithms
- Create evaluation framework
- Establish model performance benchmarks
- Build simple rule-based detection for comparison

Week 10-11: Advanced Model Development

- Implement deep learning models (LSTM, autoencoders)
- Develop ensemble methods
- Create threat classification models
- Build model versioning system
- Implement model explainability features

Week 12-13: Model Optimization

- Optimize models for performance
- Reduce false positive rates
- Implement real-time processing capabilities
- Create model validation framework
- Finalize model selection based on performance metrics

Phase 4: Response System Development (4-5 weeks)

Week 14-15: Response Framework

- Design response strategy framework
- Implement threat severity assessment
- Create automated response rules engine
- Develop network integration interfaces
- Build response logging system

Week 16-17: Advanced Response Features

- Implement reinforcement learning for response selection
- Create isolation protocols for network segments

- Develop rate limiting and traffic shaping responses
- Build integration with external security tools
- Implement response effectiveness measurement

Week 18: Feedback Mechanisms

- Create analyst feedback collection system
- Implement model update mechanisms
- Develop continuous learning pipeline
- Build model retraining triggers
- Implement A/B testing for response strategies

Phase 5: User Interface Development (3-4 weeks)

Week 19-20: Backend API Development

- Create RESTful API for system interaction
- Implement authentication and authorization
- Develop data access layers
- Build alerting system API
- Create configuration endpoints

Week 21-22: Frontend Development

- Design user dashboard layout
- Implement real-time visualization components
- Create alert management interface
- Build configuration panels
- Develop report generation system

Phase 6: System Integration & Testing (4-5 weeks)

Week 23-24: Component Integration

- Integrate all system components
- Implement message queuing for component communication
- Create system monitoring and logging
- Set up distributed deployment if needed
- Implement fault tolerance measures

Week 25-26: System Testing

- Conduct unit testing of all components

- Perform integration testing
- Run performance testing under load
- Test system against various attack scenarios
- Validate detection and response capabilities

Week 27: User Acceptance Testing

- Train test users on system operation
- Conduct usability testing
- Gather feedback on UI/UX
- Test in simulated production environment
- Document issues and prioritize fixes

Phase 7: Deployment & Documentation (3-4 weeks)

Week 28-29: Deployment Preparation

- Create deployment documentation
- Prepare production environment
- Develop deployment scripts
- Create backup and recovery procedures
- Set up monitoring for production system

Week 30-31: Final Documentation

- Complete technical documentation
- Create user manuals
- Prepare maintenance guides
- Document model training procedures
- Finalize API documentation

Phase 8: Project Completion & Future Planning (1-2 weeks)

Week 32: Project Handover

- Final system demonstration
- Knowledge transfer to operations team
- Present project findings and outcomes
- Document lessons learned
- Release final project report

Week 33: Future Planning

- Identify future enhancements
- Plan for ongoing maintenance
- Create roadmap for version 2.0
- Document research opportunities
- Close project and celebrate accomplishment

Key Milestones

1. Project plan approval (End of Phase 1)
2. Data collection infrastructure complete (Mid Phase 2)
3. Initial model performance benchmark (Mid Phase 3)
4. Response system functional (End of Phase 4)
5. User interface complete (End of Phase 5)
6. System integration complete (Mid Phase 6)
7. System deployment ready (End of Phase 7)
8. Project completion (End of Phase 8)

Resource Requirements

- **Personnel:**
 - Project Manager/Lead
 - ML Engineer(s)
 - Network Security Specialist(s)
 - Full-stack Developer(s)
 - QA Engineer(s)
- **Hardware:**
 - Development servers with GPU capabilities
 - Network monitoring hardware
 - Testing environment with network infrastructure
 - Production deployment servers
- **Software/Services:**
 - Cloud computing resources (optional)
 - ML frameworks (TensorFlow, PyTorch, etc.)
 - Network monitoring tools
 - Database systems

- Development tools and IDEs

Risk Management

Risk	Impact	Mitigation Strategy
Insufficient attack data	High	Use public datasets, synthetic data generation, simulated attacks
High false positive rate	High	Implement ensemble methods, human-in-the-loop verification, continuous model improvement
Scaling issues with traffic volume	Medium	Design for scalability, implement sampling techniques, optimize code
Integration challenges with existing network	Medium	Early stakeholder involvement, modular design, comprehensive testing
Model drift over time	Medium	Implement continuous monitoring, periodic retraining, concept drift detection